

Basics of Feature Engineering

What is a feature?

- A feature is an attribute of a data set that is used in a machine learning process.
- There is a view amongst certain machine learning practitioners that only those attributes which are meaningful to a machine learning problem are to be called as features, but this view has to be taken with a pinch of salt.
- In fact, selection of the subset of features which are meaningful for machine learning is a sub-area of feature engineering which draws a lot of research interest.
- The features in a data set are also called its dimensions.
- So a data set having 'n' features is called an n-dimensional data set.

What is a feature?

- Let's take the example of a famous machine learning data set, Iris, introduced by the British statistician and biologist Ronald Fisher.
- It has five attributes or features namely Sepal.Length, Sepal.Width, Petal.Length, Petal. Width and Species.
- Out of these, the feature 'Species' represent the class variable and the remaining features are the predictor variables.
- It is a five-dimensional data set.

What is a feature?

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
6.7	3.3	5.7	2.5	Virginica
4.9	3	1.4	0.2	Setosa
5.5	2.6	4.4	1.2	Versicolor
6.8	3.2	5.9	2.3	Virginica
5.5	2.5	4	1.3	Versicolor
5.1	3.5	1.4	0.2	Setosa
6.1	3	4.6	1.4	versicolor

What is feature engineering?

- Feature engineering refers to the process of translating a data set into features such that these features are able to represent the data set more effectively and result in a better learning performance.
- As we know already, feature engineering is an important pre-processing step for machine learning.
- It has two major elements:
 1. feature transformation
 2. feature subset selection

Feature transformation

- Feature transformation transforms the data – structured or unstructured, into a new set of features which can represent the underlying problem which machine learning is trying to solve.
- There are two variants of feature transformation:
 1. feature construction
 2. feature extraction
- Both are sometimes known as feature discovery.

Feature transformation

- **Feature construction** process discovers missing information about the relationships between features and augments the feature space by creating additional features.
- Hence, if there are 'n' features or dimensions in a data set, after feature construction 'm' more features or dimensions may get added. So at the end, the data set will become ' $n + m$ ' dimensional.
- **Feature extraction** is the process of extracting or creating a new set of features from the original set of features using some functional mapping.

Feature subset selection

Unlike feature transformation, in case of **feature subset selection** (or simply **feature selection**) no new feature is generated. The objective of feature selection is to derive a subset of features from the full feature set which is most meaningful in the context of a specific machine learning problem. So, essentially the job of feature selection is to derive a subset $F_j (F_1, F_2, \dots, F_m)$ of $F_i (F_1, F_2, \dots, F_n)$, where $m < n$, such that F_j is most meaningful and gets the best result for a machine learning problem. We will discuss these concepts in detail in the next section.

Feature Transformation

- Engineering a good feature space is a crucial prerequisite for the success of any machine learning model.
- However, often it is not clear which feature is more important.
- For that reason, all available attributes of the data set are used as features and the problem of identifying the important features is left to the learning model.
- This is definitely not a feasible approach, particularly for certain domains e.g. medical image classification, text categorization, etc.
- In case a model has to be trained to classify a document as spam or non-spam, we can represent a document as a bag of words.
- Then the feature space will contain all unique words occurring across all documents.

Feature Transformation

- This will easily be a feature space of a few hundred thousand features.
- If we start including bigrams or trigrams along with words, the count of features will run in millions.
- To deal with this problem, feature transformation comes into play.
- Feature transformation is used as an effective tool for dimensionality reduction and hence for boosting learning model performance.
- Broadly, there are two distinct goals of feature transformation:
 - Achieving best reconstruction of the original features in the data set
 - Achieving highest efficiency in the learning task

Feature construction

- Feature construction involves transforming a given set of input features to generate a new set of more powerful features.
- To understand more clearly, let's take the example of a real estate data set having details of all apartments sold in a specific region.
- The data set has three features – apartment length, apartment breadth, and price of the apartment.
- If it is used as an input to a regression problem, such data can be training data for the regression model.
- So given the training data, the model should be able to predict the price of an apartment whose price is not known or which has just come up for sale.
- However, instead of using length and breadth of the apartment as a predictor, it is much convenient and makes more sense to use the area of the apartment, which is not an existing feature of the data set.
- So such a feature, namely apartment area, can be added to the data set.
- In other words, we transform the three- dimensional data set to a four-dimensional data set, with the newly 'discovered' feature apartment area being added to the original data set.

Feature construction

apartment_ length	apartment_ breadth	apartment_ price		apartment_ length	apartment_ breadth	apartment_ area	apartment_ price
80	59	23,60,000		80	59	4,720	23,60,000
54	45	12,15,000		54	45	2,430	12,15,000
78	56	21,84,000		78	56	4,368	21,84,000
63	63	19,84,000		63	63	3,969	19,84,500
83	74	30,71,000		83	74	6,142	30,71,000
92	86	39,56,000		92	86	7,912	39,56,000

Note: Though for the sake of simplicity the features apartment length and apartment breadth have been retained but in reality, it makes more sense to exclude these features when building the model.

Feature construction

- There are certain situations where feature construction is an essential activity before we can start with the machine learning task.
- These situations are:
 - **when features have categorical value and machine learning needs numeric value inputs**
 - **when features having numeric (continuous) values and need to be converted to ordinal values**
 - **when text-specific feature construction needs to be done**

Encoding categorical (nominal) variables

- Let's take the example of another data set on athletes, as
- Say the data set has features age, city of origin, parents athlete (i.e. indicate whether any one of the parents was an athlete) and Chance of Win.
- The feature chance of a win is a class variable while the others are predictor variables.
- We know that any machine learning algorithm, whether it's a classification algorithm (like kNN) or a regression algorithm, requires numerical figures to learn from.
- So there are three features – City of origin, Parents athlete, and Chance of win, which are categorical in nature and cannot be used by any machine learning task.

Encoding categorical (nominal) variables

In this case, feature construction can be used to create new dummy features which are usable by machine learning algorithms. Since the feature 'City of origin' has three unique values namely City A, City B, and City C, three dummy features namely origin_city_A, origin_city_B, and origin_city_C is created. In the same way, dummy features parents_athlete_Y and parents_athlete_N are created for feature 'Parents athlete' and win_chance_Y and win_chance_N are created for feature 'Chance of win'. The dummy features have value 0 or 1 based on the categorical value for the original feature in that row. For example, the

Encoding categorical (nominal) variables

second row had a categorical value 'City B' for the feature 'City of origin'. So, the newly created features in place of 'City of origin', i.e. `origin_city_A`, `origin_city_B` and `origin_city_C` will have values 0, 1 and 0, respectively. In the same way, `parents_athlete_Y` and `parents_athlete_N` will have values 0 and 1, respectively in row 2 as the original feature 'Parents athlete' had a categorical value 'No' in row 2. The entire set of transformation for athletes' data set is shown in following slide

Encoding categorical (nominal) variables

Age (Years)	City of origin	Parents athlete	Chance of win
18	City A	Yes	Y
20	City B	No	Y
23	City B	Yes	Y
19	City A	No	N
18	City C	Yes	N
22	City B	Yes	Y

(a)

Age (Years)	origin_city_A	origin_city_B	origin_city_C	parents_athlete_Y	win_chance_Y
18	1	0	0	1	1
20	0	1	0	0	1
23	0	1	0	1	1
19	1	0	0	0	0
18	0	0	1	1	0
22	0	1	0	1	1

(a)

Age (Years)	origin_city_A	origin_city_B	origin_city_C	parents_athlete_Y	parents_athlete_N	win_chance_Y	win_chance_N
18	1	0	0	1	0	1	0
20	0	1	0	0	1	1	0
23	0	1	0	1	0	1	0
19	1	0	0	0	1	0	1
18	0	0	1	1	0	0	1
22	0	1	0	1	0	1	0

(b)

However, examining closely, we see that the features ‘Parents athlete’ and ‘Chance of win’ in the original data set can have two values only. So creating two features from them is a kind of duplication, since the value of one feature can be decided from the value of the other. To avoid this duplication, we can just leave one feature and eliminate the other

Encoding categorical (ordinal) variables

- Let's take an example of a student data set.
- Let's assume that there are three variable – science marks, maths marks and
- As we can see, the grade is an ordinal variable with values A, B, C, and D.
- To transform this variable to a numeric variable, we can create a feature
- num_grade mapping a numeric value against each ordinal value.
- In the context of the current example, grades A, B, C, and D in Figure 4.4a is mapped to values 1, 2, 3, and 4 in the transformed variable

Encoding categorical (ordinal) variables

marks_science	marks_maths	Grade
78	75	B
56	62	C
87	90	A
91	95	A
45	42	D
62	57	B

(a)

marks_science	marks_maths	num_grade
78	75	2
56	62	3
87	90	1
91	95	1
45	42	4
62	57	2

(b)

Transforming numeric (continuous) features to categorical features

- Sometimes there is a need of transforming a continuous numerical variable into a categorical variable.
- For example, we may want to treat the real estate price prediction problem, which is a regression problem, as a real estate price category prediction, which is a classification problem.
- In that case, we can 'bin' the numerical data into multiple categories based on the data range.
- In the context of the real estate price prediction example, the original data set has a numerical feature `apartment_price`.
- It can be transformed to a categorical variable `price-grade` either as another categorical or then further convert it into numerical.

Transforming numeric (continuous) features to categorical features

apartment_area	apartment_price
4,720	23,60,000
2,430	12,15,000
4,368	21,84,000
3,969	19,84,500
6,142	30,71,000
7,912	39,56,000

(a)

apartment_area	apartment_grade
4,720	Medium
2,430	Low
4,368	Medium
3,969	Low
6,142	High
7,912	High

(b)

apartment_area	apartment_grade
4,720	2
2,430	1
4,368	2
3,969	1
6,142	3
7,912	3

(c)

Text-specific feature construction

- In the current world, text is arguably the most predominant medium of communication.
- Whether we think about social networks like Facebook or micro-blogging channels like Twitter or emails or short messaging services such as Whatsapp, text plays a major role in the flow of information.
- Hence, text mining is an important area of research – not only for technology practitioners but also for industry practitioners.
- However, making sense of text data, due to the inherent unstructured nature of the data, is not so straightforward.
- In the first place, the text data chunks that we can think about do not have readily available features, like structured data sets, on which machine learning tasks can be executed.
- All machine learning models need numerical data as input.
- So the text data in the data sets need to be transformed into numerical features.

Text-specific feature construction

- Text data, or corpus which is the more popular keyword, is converted to a numerical representation following a process is known as vectorization.
- In this process, word occurrences in all documents belonging to the corpus are consolidated in the form of bag-of-words.
- There are three major steps that are followed:
 1. tokenize
 2. count
 3. normalize

Text-specific feature construction

- In order to tokenize a corpus, the blank spaces and punctuations are used as delimiters to separate out the words, or tokens.
- Then the number of occurrences of each token is counted, for each document.
- Lastly, tokens are weighted with reducing importance when they occur in the majority of the documents.
- A matrix is then formed with each token representing a column and a specific document of the corpus representing each row.
- Each cell contains the count of occurrence of the token in a specific document.
- This matrix is known as a document-term matrix (also known as a term-document matrix).

Text-specific feature construction

[illegible]

Feature extraction

- In feature extraction, new features are created from a combination of original features.
- Some of the commonly used operators for combining the original features include
 1. For Boolean features: Conjunctions, Disjunctions, Negation, etc.
 2. For nominal features: Cartesian product, M of N, etc.
 3. For numerical features: Min, Max, Addition, Subtraction, Multiplication, Division, Average, Equivalence, Inequality, etc.

Let's take an example and try to understand. Say, we have a data set with a feature set F_i ($F_1, F_2, \dots, F_n$). After feature extraction using a mapping function f ($F_1, F_2, \dots, F_n$) say, we will have a set of features $\dot{F}_i$ ($\dot{F}_1, \dot{F}_2, \dots, \dot{F}_m$) such that $\dot{F}_i = f(F_i)$ and $m < n$. For example, $\dot{F}_1 = k_1 F_1 + k_2 F_2$. This is depicted in

Feat _A	Feat _B	Feat _C	Feat _D		Feat ₁	Feat ₂
34	34.5	23	233		41.25	185.80
44	45.56	11	3.44		54.20	53.12
78	22.59	21	4.5	→	43.73	35.79
22	65.22	11	322.3		65.30	264.10
22	33.8	355	45.2		37.02	238.42
11	122.32	63	23.2		113.39	167.74

$$\begin{aligned} \text{Feat}_1 &= 0.3 \times \text{Feat}_A + 0.9 \times \text{Feat}_A \\ \text{Feat}_2 &= \text{Feat}_A + 0.5 \text{Feat}_B + 0.6 \times \text{Feat}_C \end{aligned}$$

Principal Component Analysis

- Every data set, as we have seen, has multiple attributes or dimensions – many of which might have similarity with each other.
- For example, the height and weight of a person, in general, are quite related.
- If the height is more, generally weight is more and vice versa.
- So if a data set has height and weight as two of the attributes, obviously they are expected to be having quite a bit of similarity.
- In general, any machine learning algorithm performs better as the number of related attributes or features reduced.
- In other words, a key to the success of machine learning lies in the fact that the features are less in number as well as the similarity between each other is very less.
- This is the main guiding philosophy of principal component analysis (PCA) technique of feature extraction.

Principal Component Analysis

- In PCA, a new set of features are extracted from the original features which are quite dissimilar in nature.
- So an n -dimensional feature space gets transformed to an m -dimensional feature space, where the dimensions are orthogonal to each other, i.e. completely independent of each other.
- To understand the concept of orthogonality, we have to step back and do a bit of dip dive into vector space concept in linear algebra.
- We all know that a vector is a quantity having both magnitude and direction and hence can determine the position of a point relative to another point in the Euclidean space (i.e. a two or three or ' n ' dimensional space)

Principal Component Analysis

A vector space is a set of vectors. Vector spaces have a property that they can be represented as a linear combination of a smaller set of vectors, called basis vectors. So, any vector 'v' in a vector space can be represented as

$$v = \sum_{i=1}^n a_i u_i$$

where,

$$a_i$$

represents 'n' scalars and

$$u_i$$

represents the basis vectors. Basis vectors are orthogonal to each other. Orthogonality of vectors in n-dimensional vector space can be thought of an extension of the vectors being perpendicular in a two-dimensional vector space. Two orthogonal vectors are completely unrelated or independent of each other. So the transformation of a set of vectors to the corresponding set of cnt..

Principal Component Analysis

- basis vectors such that each vector in the original set can be expressed as a linear combination of basis vectors helps in decomposing the vectors to a number of independent components.
- Now, let's extend this notion to the feature space of a data set.
- The feature vector can be transformed to a vector space of the basis vectors which are termed as principal components.
- These principal components, just like the basis vectors, are orthogonal to each other.
- So a set of feature vectors which may have similarity with each other is transformed to a set of principal components which are completely unrelated.
- However, the principal components capture the variability of the original feature space.
- Also, the number of principal component derived, much like the basis vectors, is much smaller than the original set of features.

Principal Component Analysis

The objective of PCA is to make the transformation in such a way that

1. The new features are distinct, i.e. the covariance between the new features, i.e. the principal components is 0.
2. The principal components are generated in order of the variability in the data that it captures. Hence, the first principal component should capture the maximum variability, the second principal component should capture the next highest variability etc.
3. The sum of variance of the new features or the principal components should be equal to the sum of variance of the original features.

Principal Component Analysis

- PCA works based on a process called eigenvalue decomposition of a covariance matrix of a data set.
- Below are the steps to be followed:
 1. First, calculate the covariance matrix of a data set.
 2. Then, calculate the eigenvalues of the covariance matrix.
 3. The eigenvector having highest eigenvalue represents the direction in which there is the highest variance. So this will help in identifying the first principal component.
 4. The eigenvector having the next highest eigenvalue represents the direction in which data has the highest remaining variance and also orthogonal to the first direction. So this helps in identifying the second principal component.
 5. Like this, identify the top 'k' eigenvectors having top 'k' eigenvalues so as to get the 'k' principal components.

Singular value decomposition

Singular value decomposition (SVD) is a matrix factorization technique commonly used in linear algebra. SVD of a matrix A ($m \times n$) is a factorization of the form:

$$A = U\Sigma V$$

where, U and V are orthonormal matrices, U is an $m \times m$ unitary matrix, V is an $n \times n$ unitary matrix and Σ is an $m \times n$ rectangular diagonal matrix. The diagonal entries of Σ are known as singular values of matrix A . The columns of U and V are called the left-singular and right-singular vectors of matrix A , respectively.

Singular value decomposition

- SVD is generally used in PCA, once the mean of each variable has been removed.
- Since it is not always advisable to remove the mean of a data attribute, especially when the data set is sparse (as in case of text data), SVD is a good choice for dimensionality reduction in those situations.
- SVD of a data matrix is expected to have the properties highlighted as follows:

Singular value decomposition

1. Patterns in the attributes are captured by the right-singular vectors, i.e. the columns of V .
2. Patterns among the instances are captured by the left-singular, i.e. the columns of U .
3. Larger a singular value, larger is the part of the matrix A that it accounts for and its associated vectors.
4. New data matrix with 'k' attributes is obtained using the equation

$$D' = D \times [v_1, v_2, \dots, v_k]$$

Thus, the dimensionality gets reduced to k . SVD is often used in the context of text data.

Linear Discriminant Analysis

- Linear discriminant analysis (LDA) is another commonly used feature extraction technique like PCA or SVD.
- The objective of LDA is similar to the sense that it intends to transform a data set into a lower dimensional feature space.
- However, unlike PCA, the focus of LDA is not to capture the data set variability.
- Instead, LDA focuses on class separability, i.e. separating the features based on class separability so as to avoid over-fitting of the machine learning model.
- Unlike PCA that calculates eigenvalues of the covariance matrix of the data set, LDA calculates eigenvalues and eigenvectors within a class and inter-class scatter matrices.
- the steps to be followed are :

1. Calculate the mean vectors for the individual classes.
2. Calculate intra-class and inter-class scatter matrices.
3. Calculate eigenvalues and eigenvectors for $S_W^{-1}S_B$, where S_W is the intra-class scatter matrix and S_B is the inter-class scatter matrix.

$$S_W = \sum_{i=1}^c S_i$$

$$S_i = \sum_{x \in D_i} (x - m_i)(x - m_i)^T$$

where m_i is the mean vector of the i -th class.

$$S_B = \sum_{i=1}^c N_i (m_i - m)(m_i - m)^T$$

where m_i is the sample mean for each class, m is the overall mean of the data set, and N_i is the sample size of each class.

4. Identify the top k eigenvectors having top k eigenvalues.

Feature Subset Selection

- Feature selection is arguably the most critical pre-processing activity in any machine learning project.
- It intends to select a subset of system attributes or features which makes a most meaningful contribution in a machine learning activity.
- Let's quickly discuss a practical example to understand the philosophy behind feature selection.
- Say we are trying to predict the weight of students based on past information about similar students, which is captured in a 'student weight' data set.
- The student weight data set has features such as Roll Number, Age, Height, and Weight.
- We can well understand that roll number can have no bearing, whatsoever, in predicting student weight.
- So we can eliminate the feature roll number and build a feature subset to be considered in this machine learning problem.
- The subset of features is expected to give better results than the full set.

Feature Subset Selection

Roll Number	Age	Height	Weight
12	12	1.1	23
14	11	1.05	21.6
19	13	1.2	24.7
32	11	1.07	21.3
38	14	1.24	25.2
45	12	1.12	23.4



Age	Height	Weight
12	1.1	23
11	1.05	21.6
13	1.2	24.7
11	1.07	21.3
14	1.24	25.2
12	1.12	23.4

Issues in high-dimensional data

- With rapid digital innovations, data generation has grown enormously, while advances in storage technology have made storing vast amounts of data inexpensive.
- This has encouraged the collection and analysis of massive, high-dimensional datasets.
- Two key domains have particularly expanded — **biomedical research**, such as gene selection from microarray data, and **text categorization**, involving huge volumes of text from social media, emails, and other sources.
- Both domains produce extremely high-dimensional data, often with tens of thousands of features.

Issues in high-dimensional data

- Extracting insights from such data poses major challenges: it demands significant computational resources and time, while model performance can degrade due to noise and overfitting.
- Moreover, models with too many features are harder to interpret.
- Thus, **feature selection** becomes essential. Its main goals are to:
 1. Build faster and more cost-effective models,
 2. Improve learning efficiency, and
 3. Enhance understanding of the underlying data-generating process.

Key Drivers of Feature Selection – Feature Relevance and Redundancy

- **Feature Relevance:** In supervised learning, each predictor variable should contribute useful information to determine the class label.
- **Irrelevant features** provide no information for prediction.
- **Weakly relevant features** contribute very little information.
- **Strongly relevant features** make a significant impact on prediction.
- In unsupervised learning, where there are no class labels, features are evaluated based on their ability to distinguish or group similar data points. Variables that do not aid in determining similarity are **irrelevant**.
- For example, in a student dataset, a student's **roll number** is irrelevant for predicting **weight** or grouping students by **academic ability**.
- Such features are removed during selection, while weakly relevant ones may be retained depending on the context.

Key Drivers of Feature Selection – Feature Relevance and Redundancy

- **Feature Redundancy:** Sometimes, multiple features carry overlapping information.
- For instance, in predicting **weight**, both **age** and **height** are correlated and provide similar insights.
- Removing one of them has little effect on the model's performance, making one feature **redundant**.
- Thus, redundant features are typically excluded, keeping only a few representative ones.
- Feature selection aims to:
 1. Eliminate irrelevant features, and
 2. Retain only representative features among redundant ones — resulting in a concise, meaningful subset tailored to the specific learning task.
- Identifying irrelevant and redundant features involves various statistical and information-based measures discussed in the following section.

Mutual Information

- As mentioned earlier, feature relevance is gauged by the amount of information contributed by a feature.
- For **supervised learning**, *mutual information* is considered a good measure of how much information a feature contributes toward determining the class label.
- Hence, it is a good indicator of the relevance of a feature with respect to the class variable.
- Higher mutual information implies higher feature relevance.

where, marginal entropy of the class,

Mutual Information

$$H(C) = - \sum_{i=1}^k p(C_i) \log_2 p(C_i)$$

Mutual information is calculated as: marginal entropy of the feature 'x',

$$\mathbf{MI(C, f) = H(C) + H(f) - H(C, f)}$$

$$H(f) = - \sum_c p(f = x) \log_2 p(f = x)$$

where

H(C) = marginal entropy of the class

H(f) = marginal entropy of the feature

C = class variable

f = feature set (taking discrete values)

K = number of classes

Entropy

- In unsupervised learning, there is **no class variable**.
Hence, feature-to-class mutual information cannot be used.
- Instead, the **entropy** of the full feature set *without one feature at a time* is calculated for all features.
- Features are then ranked in descending order of **information gain**, and the top $\beta\%$ (β is a design parameter) are selected as relevant.
- The **entropy** of a feature f is calculated using **Shannon's formula**:

$$H(f) = - \sum p(f = x) \log_2 p(f = x)$$

- This formula is used for discrete-valued features.
For continuous features, discretization is performed first to estimate the probabilities $p(f = x)$.

Measures of Feature Redundancy

- Feature redundancy refers to the **similar information contribution** by multiple features.
- There are several measures to determine redundancy, mainly:
 - **Correlation-based measures**
 - **Distance-based measures**
 - **Other coefficient-based measures**

Correlation-based Similarity Measure

- **Correlation** measures linear dependency between two random variables.
- The **Pearson correlation coefficient** is the most common measure.
- For two feature variables F_1 and F_2 , it is defined as:

$$\alpha = \frac{cov(F_1, F_2)}{\sqrt{var(F_1).var(F_2)}}$$

$$cov(F_1, F_2) = \sum (F_{1_i} - \bar{F}_1).(F_{2_i} - \bar{F}_2)$$

$$var(F_1) = \sum (F_{1_i} - \bar{F}_1)^2, \text{ where } \bar{F}_1 = \frac{1}{n} \cdot \sum F_{1_i}$$

$$var(F_2) = \sum (F_{2_i} - \bar{F}_2)^2, \text{ where } \bar{F}_2 = \frac{1}{n} \cdot \sum F_{2_i}$$

The value of r ranges from **-1 to +1**:

$\alpha = \pm 1 \rightarrow$ perfect linear relationship

$\alpha = 0 \rightarrow$ no linear relationship

In feature selection, a threshold correlation value is adopted to decide whether two features are similar enough.

Distance-based Similarity Measure

- The most common distance measure is **Euclidean distance**, Euclidean distance, which, between two features F_1 and F_2 is calculated:

$$d(F_1, F_2) = \sqrt{\sum_{i=1}^n (F_{1_i} - F_{2_i})^2}$$

- where F_1 and F_2 are feature vectors of an n -dimensional dataset.
- Lets take a data set that has two features, aptitude (F_1) and communication (F_2) under consideration.
- The Euclidean distance between the features has been calculated using the formula provided above.

Distance-based Similarity Measure

Aptitude (F_1)	Communication (F_2)	$(F_1 - F_2)$	$(F_1 - F_2)^2$
2	6	-4	16
3	5.5	-2.5	6.25
6	4	2	4
7	2.5	4.5	20.25
8	3	5	25
6	5.5	0.5	0.25
6	7	-1	1
7	6	1	1
8	6	2	4
9	7	2	4
			81.75

Distance-based Similarity Measure

A more generalized form of the Euclidean distance is the Minkowski distance, measured as :

$$d(F_1, F_2) = \left(\sum_{i=1}^n (F_{1i} - F_{2i})^r \right)^{1/r}$$

When $r = 2$, this formula represents the Euclidean distance (L_2 norm):

$$d(F_1, F_2) = \sqrt{\sum_{i=1}^n (F_{1i} - F_{2i})^2}$$

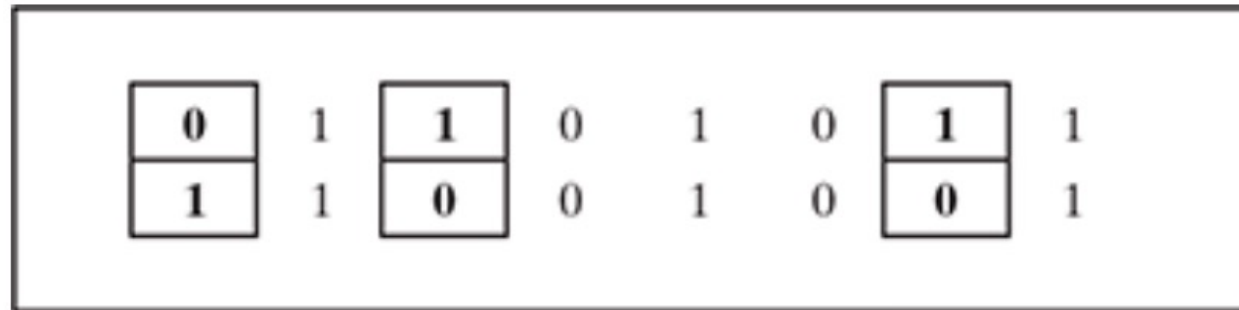
When $r = 1$, it becomes the Manhattan distance (L_1 norm):

$$d(F_1, F_2) = \sum_{i=1}^n |F_{1i} - F_{2i}|$$

Distance-based Similarity Measure

A specific example of Manhattan distance, used more frequently to calculate the distance between binary vectors is the Hamming distance.

For example, the Hamming distance between two vectors 01101011 and 11001001 is 3, as illustrated in (a)



(a) Hamming distance measurement

Other coefficient-based measures

Jaccard index/coefficient is used as a measure of similarity between two features.

The Jaccard distance, a measure of dissimilarity between two features, is complementary of Jaccard index.

For two features having binary values, Jaccard index is measured as

$$J = \frac{n_{11}}{n_{01} + n_{10} + n_{11}}$$

where,

n_{11} = number of cases where both the features have value 1

n_{01} = number of cases where feature 1 has value 0 and feature 2 has value 1

n_{10} = number of cases where feature 1 has value 1 and feature 2 has value 0

Jaccard distance is

$$d_J = 1 - J$$

Other coefficient-based measures

Simple matching coefficient (SMC) is almost same as Jaccard coefficient except the fact that it includes a number of cases where both the features have a value of 0.

$$\text{SMC} = \frac{n_{11} + n_{00}}{n_{00} + n_{01} + n_{10} + n_{11}}$$

where,

n_{11} = number of cases where both the features have value 1

n_{01} = number of cases where feature 1 has value 0 and feature 2 has value 1

n_{10} = number of cases where feature 1 has value 1 and feature 2 has value 0

n_{00} = number of cases where both the features have value 0

Other coefficient-based measures

- One more measure of similarity using similarity coefficient calculation is **Cosine Similarity**. Let's take the example of a typical text classification problem.
- The text corpus needs to be first transformed into features with a word token being a feature and the number of times the word occurs in a document comes as a value in each row.
- There are thousands of features in such a text data set.
- However, the data set is sparse in nature as only a few words do appear in a document, and hence in a row of the data set.
- So each row has very few non-zero values.
- However, the non-zero values can be anything integer value as the same word may occur any number of times.
- Also, considering the sparsity of the data set, the 0-0 matches (which obviously is going to be pretty high) need to be ignored.
- Cosine similarity which is one of the most popular measures in text classification is calculated as:

Other coefficient-based measures

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \cdot \|y\|}$$

where, $x \cdot y$ = vector dot product of x and $y = \sum_{i=1}^n x_i y_i$

$$\|x\| = \sqrt{\sum_{i=1}^n x_i^2} \quad \text{and} \quad \|y\| = \sqrt{\sum_{i=1}^n y_i^2}$$

Cosine similarity actually measures the angle between x and y vectors.

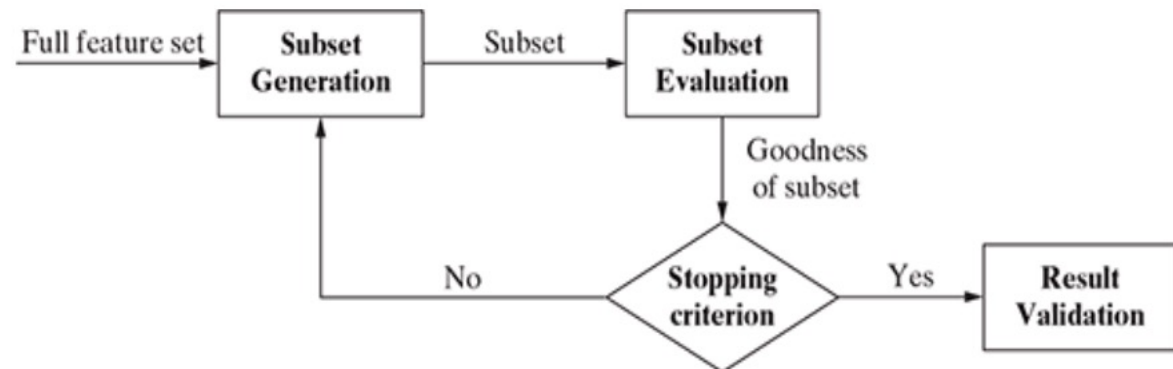
Hence, if cosine similarity has a value 1, the angle between x and y is 0° which means x and y are same except for the magnitude.

If cosine similarity is 0, the angle between x and y is 90° .

Hence, they do not share any similarity (in case of text data, no term/word is common)

Overall feature selection process

- Feature selection is the process of selecting a subset of features in a data set.
- A typical feature selection process consists of four steps:
 1. generation of possible subsets
 2. subset evaluation
 3. stop searching based on some stopping criterion
 4. validation of the result



Overall feature selection process

- For an n -dimensional dataset, 2^n subsets can theoretically be generated, but this is computationally infeasible for large n .
- Therefore, **approximate search strategies** are used:
 - **Sequential Forward Selection:** Start with an empty set and add features incrementally.
 - **Sequential Backward Elimination:** Start with all features and remove them progressively.
 - **Bi-directional Selection:** Add and remove features simultaneously.
- Each candidate subset is evaluated against the previous best using specific **evaluation criteria**. If it performs better, it replaces the prior subset.

Overall feature selection process

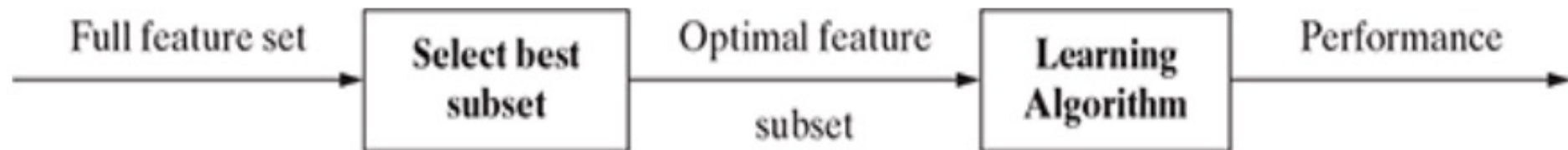
- The search stops when:
 - The search completes.
 - A set iteration or bound limit is reached.
 - Adding or deleting features no longer improves results.
 - A “good enough” subset is found (e.g., one with better classification accuracy).
- Finally, the selected subset is **validated**—in supervised learning by model accuracy, and in unsupervised learning by **cluster quality**.

Feature Selection Approaches

- There are **four main approaches** to feature selection:

1. Filter Approach:

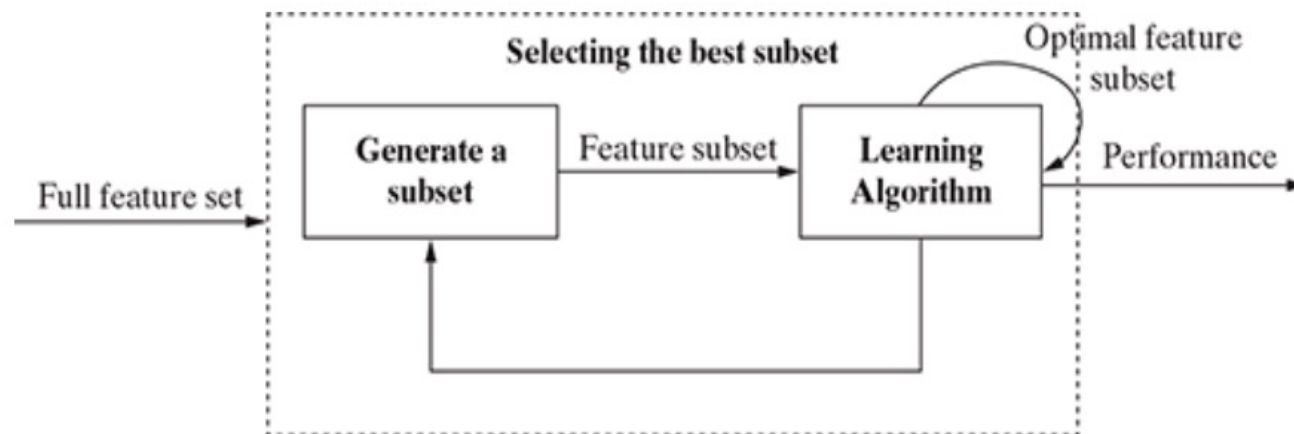
- Uses statistical tests (e.g., Pearson's correlation, Information Gain, Fisher Score, ANOVA, Chi-Square) to assess feature importance **independent of learning algorithms**.
- Fast but less accurate.



Feature Selection Approaches

2. Wrapper Approach:

- Uses a **learning algorithm as a black box** to evaluate subsets.
- Each subset is tested by training a model and checking performance.
- **More accurate** than filters but **computationally expensive**.



Feature Selection Approaches

3. Hybrid Approach:

- Combines filter and wrapper methods.
- Uses statistical tests (filter) to preselect promising subsets, then applies a learning algorithm (wrapper) to find the optimal one.

Feature Selection Approaches

4. Embedded Approach:

- Performs **feature selection and model training simultaneously** within the learning algorithm itself.
- Example: decision trees inherently perform feature selection.

